

Almost Magic Tech Lab

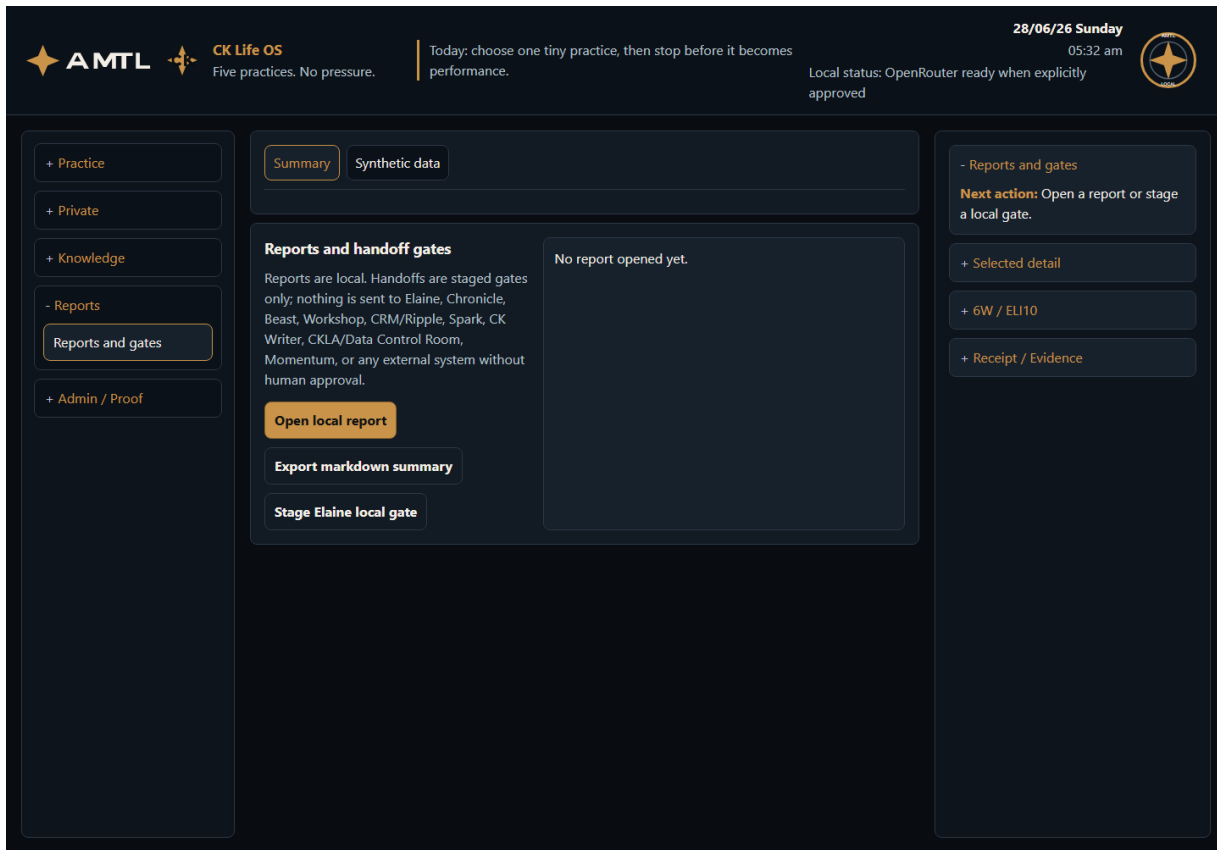
CK / Life OS

Complete Story-Flow Guide v2.0

A story-like operating map of every visible module, screen, tab, panel, button, field, backend route, data boundary, and proof path in the local/internal CK / Life OS app.

Field	Value
Product	CK / Life OS
Front door	http://amtl/ck-life-os/ and http://127.0.0.1:5420/
Runtime scope	Local/internal app, port 5420
Primary design rule	Calm, tabbed, low-pressure, neurodivergent-friendly flow
Version	v2.0
Document generated	28 June 2026
Regeneration reason	Updated after RAG / Sources source-draft changes, approved external URL fetch bo Product Bible updates, UI changes, and OpenRouter free/modest/expensive reasoni

Current calm tabbed layout screenshot



1. Opening The App

The current local/internal product does not present a public login form. The user opens the front door directly. The story begins when the user visits <http://amtl/ck-life-os/> or <http://127.0.0.1:5420/>.

Opening flow

User opens CK / Life OS -> Top bar confirms AMTL, CK Life OS, date/time, OpenRouter status, and local seal
 -> Left grouped menu appears -> Middle workspace shows the selected tabbed screen -> Right context rail shows only the relevant next action by default

Area	What the user sees	Why it matters
Top bar	AMTL logo, CK Life OS logo/name, today's signal, date/time, OpenRouter readiness, AMTL local/internal seal	Establishes identity, local runtime truth, model-provider readiness, and calm orientation.
Left menu	Practice, Private, Knowledge, Reports, Admin / Proof	Groups actions by user intent and keeps inactive areas collapsed.
Middle workspace	Tabbed screens with one visible content panel at a time	Prevents the user from scrolling through long stacked columns.

Right context rail	Next action open; Selected detail, 6W / ELI10, Receipt / Evidence collapsed	Shows relevant help without flooding the screen.
--------------------	---	--

2. Left Menu Categories

Menu category	Menu items	Story purpose
Practice	Practice desk; Guide me; Innovation lens	Daily practice and low-pressure guidance.
Private	Encrypted journal; Inner work	Private reflection, encrypted storage, and guided self-inquiry.
Knowledge	RAG / Sources	Local RAG/source status, source intake, approved NAS search, and pasted source-text search.
Reports	Reports and gates	Local summary, markdown export, staged gate, and synthetic-data checks.
Admin / Proof	Evidence summaries; Delivery matrix; R2D2 equivalent; Dependency fallback; OpenRouter routing; OSS adoption	Proof surfaces for review, kept out of normal practice flow.

Version alignment

This v2.0 guide uses the current visible UI names from index.html and the current route names from main.py. It includes the newer RAG / Sources draft/fetch behaviour and the OpenRouter free -> modest -> expensive reasoning ladder.

3. Practice Desk Story

The user opens Practice -> Practice desk. The app shows the main story panel, a practice note field, two immediate buttons, today's truth cards, and practice tabs. The user can do one small practice without being scored.

Screen element	Exact name	What happens
Main heading	Five practices for living	Frames the product as a calm practice OS.
Field	Practice note	Optional note; blank is allowed.
Button	Make today gentler	Activates Difficult Month Mode locally.

Button	Explain first step	Opens Guide me for the practice screen.
Count card	Practices	Opens Guide me for practice orientation.
Count card	Scores	Explains intentional zero.
Count card	Streaks	Explains intentional zero.
Count card	Lens items	Opens Innovation lens and loads rows.

Record a practice

User opens Practice desk -> User selects Presence / Reflection / Intention / Gratitude / Equanimity tab -> User writes optional Practice note -> User clicks Record Presence -> App calls POST /api/practices/presence/record -> Local JSONL receipt is written -> Right rail updates with next action, selected detail, 6W / ELI10, and receipt proof

Practice tab	Buttons on card	Backend routes	Result
Presence	Record Presence; Explain prompt; Details	/api/practices/presence/record; /api/prompts/after-save/presence; /api/contextual-guide/presence	Records awareness/presence receipt or explains the practice.
Reflection	Record Reflection; Explain prompt; Details	/api/practices/reflection/record; /api/prompts/after-save/reflection; /api/contextual-guide/reflection	Records contemplation/reflection receipt or explains the practice.
Intention	Record Intention; Explain prompt; Details	/api/practices/intention/record; /api/prompts/after-save/intention; /api/contextual-guide/intention	Records direction/values receipt or explains the practice.
Gratitude	Record Gratitude; Explain prompt; Details	/api/practices/gratitude/record; /api/prompts/after-save/gratitude; /api/contextual-guide/gratitude	Records appreciation receipt or explains the practice.
Equanimity	Record Equanimity; Explain prompt; Details	/api/practices/equanimity/record; /api/prompts/after-save/equanimity; /api/contextual-guide/equanimity	Records steadiness/balance receipt or explains the practice.

What the user learns

CK / Life OS is deliberately not a scorekeeper. The zero score and zero streak cards are explanatory proof, not missing data.

4. Guide Me Story

The user opens Practice -> Guide me. The screen has two tabs: Start and Practice 6W.

Tab or button	Exact name	What the app does
Tab	Start	Shows screen-specific guide choices.
Tab	Practice 6W	Shows each practice with contextual What / Who / Why / Wh / Where / ELI10.
Button	Practice	Calls /api/guided-use?view=practice.
Button	Ideas	Calls /api/guided-use?view=ideas.
Button	Reports	Calls /api/guided-use?view=reports.
Button	Evidence	Calls /api/guided-use?view=evidence.

Tired-user guide flow

User opens Guide me -> User clicks Practice / Ideas / Reports / Evidence -> App returns start_here, next_click, evidence_to_check, do_not_do_yet, write_boundary -> Right rail updates with the selected guide

5. Innovation Lens Story

The user opens Practice -> Innovation lens. The lens turns the 500 generated field/navigation innovation patterns into calm, contextual helpers instead of a raw register.

Tab	Fields or buttons	Story
Filter	Practice; Situation; Open matching innovations	The user narrows the 500 patterns to a calm subset.
Results	Open {idea_id}	The app shows matching rows and lets the user open one pattern.
Detail	Usefulness, control, cost, calm design, neurodivergent support, 6W / ELI10	The user sees why the pattern matters and whether to use it.

Innovation lens flow

User chooses Practice and Situation -> User clicks Open matching innovations -> App calls /api/ideas ->

Results tab opens -> User clicks Open {idea_id} -> App calls /api/ideas/{idea_id} -> Detail tab opens with 6W / ELI10 and synthetic boundary

Situation filter	Meaning in the story
morning	Start the day gently.
midday	Recover attention or reduce friction during the day.
evening	Reflect without turning the day into performance.
difficult-day	Use gentler supports when pressure is high.
low-energy	Reduce effort and choice load.
family	Make practice useful around relationships/family.
work	Use the practice inside work context.
body	Bring attention back to body and energy.
creative	Support creativity without clutter.
repair	Help after a rupture or difficult moment.

6. Encrypted Journal Story

The user opens Private -> Encrypted journal. The screen has tabs: Write, Entries, and Detail.

Tab	Fields/buttons	What happens
Write	Practice; Title; Encrypted journal entry; Save encrypted entry; Refresh journal	User writes a private note. Save encrypts title/content with Fernet in the local runtime folder.
Entries	Open local decrypt	User sees encrypted entries and can open one locally.
Detail	Decrypted content	The app decrypts only for the local UI and shows the note.

Journal save and read flow

User writes in Encrypted journal entry -> User clicks Save encrypted entry -> App calls POST /api/journal -> Encrypted JSONL row is written -> User clicks Refresh journal -> Entries tab lists local encrypted entries ->

User clicks Open local decrypt -> Detail tab shows readable content inside local UI only

7. Inner Work Story

The user opens Private -> Inner work. This is for shadow integration, observation / Krishnamurti-style inquiry, grounding, relationship patterns, and unknown-mode reflection. It guides looking; it does not diagnose, therapise, or claim spiritual authority.

Tab	Fields/buttons	What happens
Start	Mode; Intention; Inner work text input; Voice transcript input; Start guided session; Load sessions	User chooses depth and writes text or pasted voice transcript. The app returns a posture, first question, and seven-step program.
Sessions	Open session	User loads encrypted session history and opens a selected session.
Detail	Guidance detail	Shows mode label, posture, not-answer boundary, first question, safety state, program, controls, AI provider called false.

Mode	Story use
Shadow integration	Meet a rejected/protective part without acting it out or shaming it.
Observation / Krishnamurti-style inquiry	Observe thought, fear, image, comparison, and becoming without authority claims.
Grounding first	Stabilise before depth; choose safety over insight.
Relationship pattern	Separate event, story, need, and next clean action.
I do not know yet	Start gently and choose the right depth after the first reflection.

Inner Work safety flow

User writes text -> App checks for risk language -> If risk language appears, guidance switches to Grounding first -> Depth becomes grounding_only -> Session is encrypted locally -> External send and provider call remain false

8. RAG / Sources Story

The user opens Knowledge -> RAG / Sources. This is where RAG lives: approved internal/NAS source search, local source drafts, pasted source text, and explicitly approved one-time external URL fetches into local RAG. The app does not fetch a URL without the approval control.

Tab	Fields/buttons	What happens
RAG status	Explain RAG access; Refresh source status; Build local index	Shows RAG location, internal/external lanes, approved roots, exact two exclusions, provider-call false, and pgvector-not-required truth.
Add source	Source type; Title; Path or URL; RAG source notes; RAG source text; Approve one-time external URL fetch into local RAG; Stage source for approval	Stages an internal path or external reference locally. Optional pasted source text becomes searchable local RAG material. External URLs fetch into local RAG only when the one-time approval checkbox is selected.
Search	Search sources; Search	User searches combined local RAG drafts and approved NAS source index.
Results	Source matches	Shows draft or NAS-index matches, local paths/references, and safe snippets.

Approved NAS exclusions

\\NAS2\amtl-documents\escalation-logs\linux logs\wazuz\prefe
 \\Nas1\Nas\Programs\Spiritual\Dr Joe Dispenza\Generous moment\mani

RAG / Sources flow

User opens RAG / Sources -> User clicks RAG status -> App calls /api/rag/status -> User may stage internal/external source reference -> App calls POST /api/rag/source-draft -> Pasted source text is saved locally for search -> User may build local NAS index -> App calls POST /api/source-index/build -> User searches -> App calls /api/rag/search -> Results tab shows local draft and NAS matches

9. Reports And Gates Story

The user opens Reports -> Reports and gates. The section has Summary and Synthetic data tabs.

Tab	Button	Backend route	User result
Summary	Open local report	/api/reports/local-summary	Shows report ID, receipt count,

			encrypted journal count, innovation count, external-send false, source-write false.
Summary	Export markdown summary	/api/reports/local-summary.md	Shows copy-ready markdown in the local UI.
Summary	Stage Elaine local gate	/api/handoffs/lifeos-elaine-local/stage	Creates a local staged receipt; nothing is sent to Elaine.
Synthetic data	Check boundary	/api/synthetic-data/status	Shows synthetic row status and boundary truth.
Synthetic data	Preview cleanup	/api/synthetic-data/removal-preview	Shows what would be removed; currently zero operational user history.
Synthetic data	Run cleanup verification	/api/synthetic-data/cleanup	Creates local verification receipt proving synthetic boundary.

10. Evidence Summaries Story

The user opens Admin / Proof -> Evidence summaries. Proof is not shown in normal practice flow; it lives behind Checks and Output tabs.

Button	Route	What it proves
Product Bible matrix	/api/product-bible-matrix	Feature coverage, delivery matrix, source index, and scope truth.
R2D2 equivalent	/api/r2d2	Requirement-to-delivery audit checks.
Dependency fallback	/api/dependency-status	Core UI does not require Postgres, scheduler, provider, source index, or unrelated apps.
Data truth	/api/data-truth	Receipts, encrypted journal, inner work, innovation catalogue, NAS source index, external-send/source-write/provider-call truth.
Button/drilldown truth	/api/ui-truth	Buttons, drilldowns, counts, and UI truth.
OSS adoption	/api/oss-adoption	Open-source dependency truth.
Cost and effort reduction	/api/cost-effort-reduction	How the app reduces decision/setup/rework burden.
Field utilities	/api/field-utilities	Field-level helpers and drilldowns.
OpenRouter model routing	/api/ai/model-policy	OpenRouter free/modest/expensive model policy,

		auto reasoning, and paid-tier approval boundary.
Source index	/api/source-index/status	NAS source index status and approved exclusions.

11. OpenRouter Reasoning Ladder Story

The user does not normally start on an AI provider screen. The OpenRouter path appears through Admin / Proof -> OpenRouter routing or through any future AI feature that needs model routing. The app first explains which tier it would use, then only performs a live provider call when explicit execution and approval gates are satisfied.

Tier	Visible label	Reasoning level	Default model	Approval story
free	Free models first	low / routine / cheap	openrouter/free	Preferred first path. A live call still requires OPENROUTER_API_KEY and execute=true, but it does not require paid-tier approval.
modest	A little bit expensive	medium / normal / moderate	openai/gpt-5.1	Used when the task needs stronger reasoning than the free tier. Live call requires execute=true and allow_paid_provider=true.
expensive	Full expensive models	high / deep / hard	openai/gpt-5.5	Reserved for deep audits, architecture, privacy/security, root cause work, product-bible synthesis, or high-value reasoning. Live call requires execute=true and allow_paid_provider=true.

OpenRouter route-only flow

User clicks Admin / Proof -> OpenRouter routing -> App calls /api/ai/model-policy -> User or feature sends task text to /api/ai/route -> App infers free, modest, or expensive from reasoning level and task keywords -> UI shows selected model, paid status, and approval requirement -> No provider call occurs

OpenRouter live-call flow

User or future feature asks for AI completion -> App calls /api/ai/complete -> If execute is not true, app returns route recommendation only -> If selected tier is modest or expensive, allow_paid_provider must be true -> OPENROUTER_API_KEY stays server-side -> Only then does app call OpenRouter -> Response returns with provider_called true and selected tier proof

AI provider boundary

OpenRouter is the chosen AI provider for this product path. The current app keeps local/internal features usable without any live provider call, and the UI presents model policy as proof before execution.

12. Runtime, Backend, And Data Story

Runtime item	Value / route	Story truth
Direct URL	http://127.0.0.1:5420/	Local app front door.
AMTL URL	http://amtl/ck-life-os/	Friendly gateway route.
Alias	http://amtl/lifeos/	Friendly alias route.
Health	/health and /api/health	Confirms runtime ok, port 5420, no gamification, runtime independence.
Practice receipts	Local JSONL	Local practice/system receipts, no external send.
Encrypted journal	Local encrypted JSONL	Private title/content encrypted at rest.
Inner Work	Local encrypted JSONL	Private guided sessions encrypted at rest.
Innovation catalogue	Generated local catalogue	500 field/navigation patterns; synthetic, not lived history.
RAG source drafts	Local JSONL	Internal/external source references plus optional pasted source text; no external fetch.
NAS source index	Local metadata/text-snippet JSONL	Internal source truth with exactly two approved exclusions.
OpenRouter	/api/ai/model-policy; /api/ai/route; /api/ai/complete	OpenRouter free first, modest paid, then full expensive. Live calls require key and execute=true; modest/full tiers also require allow_paid_provider=true.

Generic click-to-proof flow

User clicks menu item -> App opens screen -> User clicks tab/button or enters field -> App calls backend route -> Local data/proof/result is generated -> Middle tab updates -> Right context rail updates -> Evidence remains local unless explicitly approved elsewhere

13. Dependency-Down And Boundary Story

If PostgreSQL, scheduler, OpenRouter, NAS, Workshop, Beast, or another AMTL product is down, the core Practice desk, Guide me, Encrypted journal, Inner work, Reports, and Evidence screens remain usable.

Optional dependencies are labelled as optional, fallback, or unavailable; they are not hidden as fake success.

Dependency	Core required?	Fallback story
PostgreSQL / pgvector	No	Core uses local JSONL and generated/local data. RAG source search uses local drafts and NAS index, not pgvector-required
Scheduler	No	Manual practice recording remains available.
OpenRouter	No	Local fallback explains routing. Provider_called remains false without key/execution/approval.
NAS source index	No for core practice UI	RAG / Sources screen can show not_built or access issues; practice flow remains usable.
Other AMTL apps	No	Handoffs are local staged gates only.

14. Complete Screen And Button Checklist

Section	Tabs / panels	Buttons / fields covered
Top bar	AMTL logo, CK Life OS logo/name, today's signal, date/time/status, AMTL seal	No buttons; orientation only.
Practice desk	Practice tabs; Today's truth	Practice note; Make today gentler; Explain first step; Practices; Scores; Streaks; Lens items; Record; Explain prompt; Details.
Guide me	Start; Practice 6W	Practice; Ideas; Reports; Evidence.
Innovation lens	Filter; Results; Detail	Practice; Situation; Open matching innovations; Open {idea_id}.
Encrypted journal	Write; Entries; Detail	Practice; Title; Encrypted journal entry; Save encrypted entry; Refresh journal; Open local decrypt.
Inner work	Start; Sessions; Detail	Mode; Intention; Inner work text input; Voice transcript input; Start guided session; Load sessions; Open session.
RAG / Sources	RAG status; Add source; Search; Results	Explain RAG access; Refresh source status; Build local index; Source type; Title; Path or URL; RAG

		source notes; RAG source text; Approve one-time external URL fetch into local RAG; Stage source for approval; Search sources; Search.
Reports and gates	Summary; Synthetic data	Open local report; Export markdown summary; Stage Elaine local gate; Check boundary; Preview cleanup; Run cleanup verification.
Evidence summaries	Checks; Output	Product Bible matrix; R2D2 equivalent; Dependency fallback; Data truth; Button/drilldown truth; OSS adoption; Cost and effort reduction; Field utilities; OpenRouter model routing; Source index.
OpenRouter routing	Admin / Proof direct button; Evidence summaries proof button	Free models first; A little bit expensive; Full expensive models; auto reasoning; execute=true; allow_paid_provider=true for paid tiers; server-side OPENROUTER_API_KEY boundary.
Right context rail	Next action; Selected detail; 6W / ELI10; Receipt / Evidence	Collapsed disclosure panels; updates when the user clicks.

Plain-English ending

CK / Life OS is a calm local/internal practice operating system. The user can practice, reflect privately, do Inner Work, use innovation helpers, inspect sources, create local reports, and review proof without the app becoming noisy, performative, or externally mutating anything.